

Reviewer Pack — Minimal p-Adic Divergence and Communication Complexity

Entry e27d63309b42 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 07:13:38 UTC

Minimal p-Adic Divergence and Communication Complexity

Entry ID: e27d63309b42 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 07:13:38 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Number Theory **Field B** (complexity object): Communication Complexity

Statement:

The minimal p-adic divergence between the indicator functions of two independent random variables, each being a bit in a communication protocol, is $O(\log n)$ where n is the number of bits.

Rationale (proposer's reasoning):

p-Adic numbers provide a non-Archimedean valuation that can capture subtle properties of probability distributions. This conjecture suggests that p-adic divergence could be used to quantify complexity lower bounds for communication protocols, offering a novel perspective on information transfer.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d0c81cf988e1e06b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a communication protocol with n bits, the minimal p-adic divergence between indicator functions of two independent random variables is $O(\log n)$ if the mean of the p-adic divergences across 30 seeds is less than or equal to $\log_2(n)$

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic divergence AND Communication Complexity - indicator function p-adic AND protocol complexity - log n p-adic distance AND information theory

Top relevant hits considered: - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/hep-th/9410058v3] p-Adic description of Higgs mechanism I: p-Adic square root and p-adic light cone - [http://arxiv.org/abs/1309.3982v3] Desingularization of complex multiple zeta-functions, fundamentals of p -adic multiple L -functions, and evaluation of - [http://arxiv.org/abs/2201.08874v2] Note on p -adic Local Functional Equation - [http://arxiv.org/abs/hep-th/0312046v1] p-Adic and Adelic Quantum Mechanics - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def p_adic_divergence(p, a, b):
    pa = [p**i * (a % p) for i in range(len(a))]
    pb = [p**i * (b % p) for i in range(len(b))]

    def hamming_distance(x, y):
        return sum(xi != yi for xi, yi in zip(x, y))

    max_length = max(len(pa), len(pb))
    pa += [0] * (max_length - len(pa))
    pb += [0] * (max_length - len(pb))

    return hamming_distance(pa, pb)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    metric_values = []
    instances_tested = 0
    n_max = 0

    for n in n_values:
        a = [random.randint(0, 1) for _ in range(n)]
        b = [random.randint(0, 1) for _ in range(n)]
```

```

        divergence = p_adic_divergence(2, a, b)
        metric_values.append(divergence)
        instances_tested += n
        n_max = max(n_max, n)

    mean_value = sum(metric_values) / len(metric_values)
    conjecture_holds = mean_value <= math.log2(n_max)
    counterexample = "" if conjecture_holds else f"mean={mean_value} > log2({n_max})"

    return {
        "metric_name": "p-adic divergence",
        "metric_value": mean_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mean exceeds log2(n)\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_26a6ff49.py", line 66, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_26a6ff49.py", line 42, in run_trial
    divergence = p_adic_divergence(2, a, b)
                ^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_26a6ff49.py", line 19, in p_adic_divergence

```

```
pa = [p**i * (a % p) for i in range(len(a))]
```

TypeError: unsupported operand type(s) for %: 'list' and 'int'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution due to a TypeError, which prevented it from producing any data or results. | next: Debug the test code to fix the TypeError and rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18455
2	propose	ollama_remote	glm4:latest	0	0	18627
3	preregistration	ollama_remote	glm4:latest	0	0	11774
4	novelty	ollama_remote	glm4:latest	0	0	7898
5	novelty	ollama_remote	glm4:latest	0	0	9873
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11905
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7962
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9502
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8133
10	judge	ollama_remote	glm4:latest	0	0	11637

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 115765 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/e27d63309b42.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/e27d63309b42.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/e27d63309b42.tar.gz* (if generated)